Laboratório Prático Integrado

Do problema real ao banco de dados implementado e consultado · Caso: Cafeteria & Grãos Artesanais — Sabor & Ponto

Grupo	7
Integrantes	_____
SGBD	PostgreSQL 16 — script completo em `laboratorio_sabor_ponto.sql`
Data	___/___/2026 — preenchido em 30/10/2026

> **Observação:** o script foi executado do início ao fim em um banco vazio. Os resultados e as mensagens de erro abaixo foram copiados da saída do PostgreSQL sem alteração.

Bloco 1 — Negócio, regras e DER

Entidades e atributos

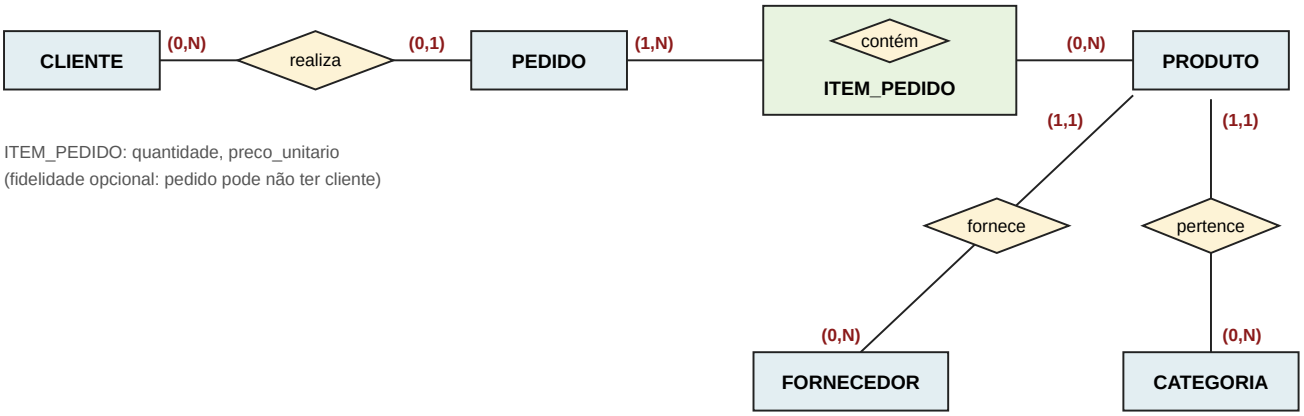
Entidade	Atributos principais	Regra
CLIENTE	id_cliente, nome, cpf, telefone, email	1 — programa de fidelidade
PEDIDO	id_pedido, data_pedido, valor_total (derivado)	2 — data e valor total
PRODUTO	id_produto, nome, preco	4, 5
CATEGORIA	id_categoria, nome	4 — Grãos, Lanches, Bebidas Quentes
FORNECEDOR	id_fornecedor, nome, cnpj	5 — fornecedor principal
ITEM_PEDIDO (associativa)	quantidade, preco_unitario	3 — produto e quantidade vendida

As **4 principais** são Cliente, Pedido, Produto e Categoria. Mas a regra 5 dá ao fornecedor atributos próprios (nome e CNPJ), então ele também é entidade; e a regra 3 cria um N:N entre Pedido e Produto, resolvido pela entidade associativa ITEM_PEDIDO.

Relacionamentos e cardinalidades

Relacionamento	Leitura
CLIENTE (0,N) — realiza — (0,1) PEDIDO	Um cliente faz 0 ou vários pedidos; um pedido tem no máximo um cliente — pode não ter, porque o cadastro de fidelidade é opcional.
PEDIDO (1,N) — contém — (0,N) PRODUTO	Um pedido tem 1 ou mais produtos; um produto aparece em 0 ou vários pedidos. N:N → ITEM_PEDIDO.
PRODUTO (1,1) — pertence — (0,N) CATEGORIA	Todo produto pertence a exatamente uma categoria.
PRODUTO (1,1) — é fornecido por — (0,N) FORNECEDOR	Cada produto tem um único fornecedor principal.

Desenho conceitual



ITEM_PEDIDO: quantidade, preco_unitario
(fidelidade opcional: pedido pode não ter cliente)

Bloco 2 — Modelo lógico e normalização

Tabela	Colunas (PK sublinhada · FK em itálico)
categoria	id_categoria, nome
fornecedor	id_fornecedor, nome, cnpj
produto	id_produto, nome, preco, *id_categoria*, *id_fornecedor*
cliente	id_cliente, nome, cpf, telefone, email
pedido	id_pedido, data_pedido, *id_cliente* (aceita nulo)
item_pedido	*id_pedido*, *id_produto*, quantidade, preco_unitario

Relacionamentos 1:N viram FK no lado N (produto recebe id_categoria e id_fornecedor; pedido recebe id_cliente). **O N:N** virou a tabela `item_pedido`, com **chave composta** (id_pedido, id_produto): o mesmo produto só aparece uma vez em cada pedido; se o cliente leva dois, isso vai na quantidade.

valor_total não é coluna: depende dos itens, e não da chave do pedido (seria dependência transitiva, contra a 3FN). É calculado pela visão `v_pedido_total`.

Por que ItemPedido está na 2FN

A chave é composta: (id_pedido, id_produto). A 2FN exige que todo atributo não chave dependa da **chave toda**. `quantidade` é a quantidade daquele produto naquele pedido; `preco_unitario` é o preço praticado daquele produto naquele pedido (o preço do catálogo pode mudar depois). Os dois dependem de (id_pedido, id_produto) juntos. Atributos que dependeriam de só uma parte — como o nome do produto (id_produto → nome) ou a data do pedido (id_pedido → data_pedido) — ficaram nas tabelas `produto` e `pedido`. Não há dependência parcial.

Bloco 3 — Projeto físico: DDL e DML

Script DDL

```
CREATE TABLE categoria (
    id_categoria  INTEGER      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome          VARCHAR(60)  NOT NULL UNIQUE
);

CREATE TABLE fornecedor (
    id_fornecedor INTEGER      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome          VARCHAR(120) NOT NULL,
    cnpj          CHAR(14)     NOT NULL UNIQUE CHECK (cnpj ~ '^[0-9]{14}$')
);

CREATE TABLE produto (
    id_produto    INTEGER       GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome          VARCHAR(120)  NOT NULL,
    preco         NUMERIC(10,2) NOT NULL CHECK (preco > 0),
    id_categoria  INTEGER       NOT NULL REFERENCES categoria (id_categoria),
    id_fornecedor INTEGER       NOT NULL REFERENCES fornecedor (id_fornecedor)
);

CREATE TABLE cliente (
    id_cliente    INTEGER      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome          VARCHAR(120) NOT NULL,
    cpf           CHAR(11)     NOT NULL UNIQUE CHECK (cpf ~ '^[0-9]{11}$'),
    telefone      VARCHAR(20),
    email         VARCHAR(120)
);

CREATE TABLE pedido (
    id_pedido     INTEGER       GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    data_pedido   TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_cliente    INTEGER       REFERENCES cliente (id_cliente)   -- NULL: venda sem fidelidade
);

CREATE TABLE item_pedido (
    id_pedido      INTEGER       NOT NULL REFERENCES pedido (id_pedido),
    id_produto     INTEGER       NOT NULL REFERENCES produto (id_produto),
    quantidade     INTEGER       NOT NULL CHECK (quantidade > 0),
    preco_unitario NUMERIC(10,2) NOT NULL CHECK (preco_unitario > 0),
    PRIMARY KEY (id_pedido, id_produto)
);

-- valor total do pedido: derivado dos itens (3FN), por isso é uma visão e não uma coluna
CREATE VIEW v_pedido_total AS
SELECT p.id_pedido, p.data_pedido, p.id_cliente,
       COALESCE(SUM(i.quantidade * i.preco_unitario), 0) AS valor_total
FROM pedido p LEFT JOIN item_pedido i ON i.id_pedido = p.id_pedido
GROUP BY p.id_pedido, p.data_pedido, p.id_cliente;
```

Restrições: **PK** e **FK** em todas as tabelas; **NOT NULL** em nome do produto, preço, data do pedido, quantidade; **CHECK** para preço e quantidade maiores que zero; **UNIQUE** no CPF, no CNPJ e no nome da categoria.

Script DML (massa de dados)

```
INSERT INTO categoria (nome) VALUES ('Grãos'), ('Lanches'), ('Bebidas Quentes'), ('Métodos de
Extração');

INSERT INTO fornecedor (nome, cnpj) VALUES
('Fazenda Alta Mogiana Cafés Especiais', '11222333000144'),
('Padaria Artesanal Trigo Bom', '55666777000188'),
('Barista Utensílios LTDA', '99888777000166');

INSERT INTO produto (nome, preco, id_categoria, id_fornecedor) VALUES
('Café Etiópia 250g', 54.90, 1, 1),
('Café Bourbon Amarelo 250g', 42.00, 1, 1),
('Pão de Queijo Recheado', 9.50, 2, 2),
('Croissant de Amêndoas', 16.00, 2, 2),
('Espresso Duplo', 11.00, 3, 1),
('Cappuccino Artesanal', 15.50, 3, 1),
('Prensa Francesa 350ml', 129.90, 4, 3);

INSERT INTO cliente (nome, cpf, telefone, email) VALUES
('Juliana Prado', '12312312300', '11987654321', 'juliana.prado@email.com'),
('Marcos Vinicius Teles', '45645645600', '11976543210', 'marcos.teles@email.com');

INSERT INTO pedido (data_pedido, id_cliente) VALUES
('2026-10-26 08:15', 1),
('2026-10-26 09:40', 2),
('2026-10-27 16:05', 1),
('2026-10-27 17:30', NULL);      -- cliente sem cadastro de fidelidade

INSERT INTO item_pedido (id_pedido, id_produto, quantidade, preco_unitario) VALUES
(1, 1, 1, 54.90), (1, 5, 1, 11.00),
(2, 6, 2, 15.50), (2, 3, 2, 9.50),
(3, 7, 1, 129.90), (3, 2, 1, 42.00),
(4, 4, 1, 16.00);
```

4 categorias, 3 fornecedores, 7 produtos, 2 clientes e 4 pedidos — um deles (pedido 4) sem cadastro de fidelidade.

Testes de integridade (todos devem ser recusados)

```
[T1] preço negativo (CHECK)
ERROR:  new row for relation "produto" violates check constraint "produto_preco_check"
DETAIL:  Failing row contains (8, Teste, -5.00, 1, 1).
[T2] item com produto inexistente (FK)
ERROR:  insert or update on table "item_pedido" violates foreign key constraint
"item_pedido_id_produto_fkey"
DETAIL:  Key (id_produto)=(99) is not present in table "produto".
[T3] mesmo produto duas vezes no pedido (PK composta)
ERROR:  duplicate key value violates unique constraint "item_pedido_pkey"
DETAIL:  Key (id_pedido, id_produto)=(1, 1) already exists.
[T4] CPF repetido (UNIQUE)
ERROR:  duplicate key value violates unique constraint "cliente_cpf_key"
DETAIL:  Key (cpf)=(12312312300) already exists.
[T5] quantidade zero (CHECK)
ERROR:  new row for relation "item_pedido" violates check constraint
"item_pedido_quantidade_check"
DETAIL:  Failing row contains (4, 5, 0, 11.00).
```

Bloco 4 — Consultas com SELECT e JOIN

1. Produtos acima de R$ 15,00, do mais caro para o mais barato

```
SELECT nome, preco
FROM produto
WHERE preco > 15.00
ORDER BY preco DESC;
```

```
[C1] Produtos acima de R$ 15,00, do mais caro para o mais barato
           nome            | preco
---------------------------+--------
 Prensa Francesa 350ml     | 129.90
 Café Etiópia 250g         |  54.90
 Café Bourbon Amarelo 250g |  42.00
 Croissant de Amêndoas     |  16.00
 Cappuccino Artesanal      |  15.50
(5 rows)
```

2. Produto e categoria (INNER JOIN, relacionamento 1:N)

```
SELECT p.nome AS produto, c.nome AS categoria
FROM produto p
INNER JOIN categoria c ON c.id_categoria = p.id_categoria
ORDER BY c.nome, p.nome;
```

```
[C2] Produto e categoria (INNER JOIN)
          produto          |      categoria
---------------------------+---------------------
 Cappuccino Artesanal      | Bebidas Quentes
 Espresso Duplo            | Bebidas Quentes
 Café Bourbon Amarelo 250g | Grãos
 Café Etiópia 250g         | Grãos
 Croissant de Amêndoas     | Lanches
 Pão de Queijo Recheado    | Lanches
 Prensa Francesa 350ml     | Métodos de Extração
(7 rows)
```

3. Pedido, data, cliente e valor total (JOIN múltiplo)

```
SELECT t.id_pedido, t.data_pedido, c.nome AS cliente, t.valor_total
FROM v_pedido_total t          -- pedido + soma de item_pedido
INNER JOIN cliente c ON c.id_cliente = t.id_cliente
ORDER BY t.id_pedido;
```

```
[C3] Pedido, data, cliente e valor total
 id_pedido |     data_pedido     |        cliente        | valor_total
-----------+---------------------+-----------------------+-------------
         1 | 2026-10-26 08:15:00 | Juliana Prado         |       65.90
         2 | 2026-10-26 09:40:00 | Marcos Vinicius Teles |       50.00
         3 | 2026-10-27 16:05:00 | Juliana Prado         |      171.90
(3 rows)
```

O valor total sai da junção de `pedido` com `item_pedido` (visão `v_pedido_total`). Com `INNER JOIN`, o pedido 4, sem cliente, não aparece. Se a gerência quiser ver **todas** as vendas, usa `LEFT JOIN`:

```
[C3b] Variação com LEFT JOIN: inclui a venda sem cadastro de fidelidade
 id_pedido |     data_pedido     |        cliente        | valor_total
-----------+---------------------+-----------------------+-------------
         1 | 2026-10-26 08:15:00 | Juliana Prado         |       65.90
         2 | 2026-10-26 09:40:00 | Marcos Vinicius Teles |       50.00
         3 | 2026-10-27 16:05:00 | Juliana Prado         |      171.90
         4 | 2026-10-27 17:30:00 | (sem cadastro)        |       16.00
(4 rows)
```

Encerramento

O caso percorreu o caminho completo: regras de negócio → DER → tabelas com PK e FK → normalização → DDL com restrições → dados → consultas para a gerência. As restrições recusaram os 5 dados inválidos testados, e as consultas respondem às três perguntas da cafeteria. O mesmo caminho foi seguido no projeto do grupo (ERP Petly Club).